🎄 Navidad en Rojo y Verde 2026

Arquitectura de Sistemas Distribuidos, TDD y Ecosistema Full-Stack

Gimnasio de TDD en Java 21: 24 retos que empiezan en rojo y terminan en verde, más una plataforma distribuida de alta fiabilidad.

Autoría: **Equipo Navidad en Rojo y Verde**

`github.com/martillohamer/navidad-en-rojo-y-verde-2026`

📊 Resumen ejecutivo

La plataforma evolucionó de un calendario de 24 retos de lógica a un **ecosistema distribuido completo**, sin salir nunca del ciclo **TDD**.

- 🟢 **100 % TDD**: cada componente nació como un test en rojo.
- ✅ **256 tests en verde absoluto** (`BUILD SUCCESS` en modo reto y soluciones).
- 🔁 **50 ficheros emparejados** con simetría binaria `src/main` ↔ `src/solutions` .
- 🧱 **Cero dependencias externas**: solo Java 21, Maven y JUnit 5.
- 🧩 **13 subsistemas** en `reto.extension.*` (`+147 tests` sobre el calendario base).

🏛️ Pilares del repositorio

1. Los 24 retos base — de "El despertar" a "El simulador del gran despegue": cadenas, colecciones, streams, records y orquestación de clases. **77 tests**.

2. Extensiones de alta fiabilidad — sistemas de nivel industrial:

- **BFT**: consenso tolerante a fallos bizantinos sobre red P2P (`GrinchChaosEngine`).
- **Merkle Ledger**: ledger inmutable con SHA-256 (`MerkleLedger`).
- **WebSocket**: telemetría en tiempo real RFC 6455 (`reto.extension.websocket`).

🛰️ Subsistema WebSocket

`reto.extension.websocket` expone telemetría de agentes en tiempo real bajo el endpoint `/ws/telemetry`.

- **Servidor nativo en Java puro**: sin Spring, sin Netty, sin librerías externas.
- **RFC 6455 implementado a mano**: handshake `Sec-WebSocket-Accept` = `Base64(SHA-1(key + GUID))` y codificación de *text frames* (`0x81`, longitud corta y extendida).
- **Arquitectura limpia**: `TelemetryMessage` (`record` inmutable con JSON propio), `TelemetryService` (flujo simulado determinista) y `TelemetryWebSocketServer`.

⌨ Herramienta CLI

`reto.extension.cli.ConsolaNexus` es una consola interactiva que orquesta los motores del ecosistema.

- **Argumentos inmutables**: `record ArgumentosCli` y `ComandoCli`, con parser propio de `--clave` / `--clave=valor`.
- **Orquestación de motores**: los comandos llaman a los motores **reales** (`GrinchChaosEngine`, `MerkleLedger`, `TelemetryService`).
- **Patrón REPL**: bucle `nexus>` con parada en `exit`, con `PrintStream` **inyectado** para testear sin tocar `System.out`.

Comando	Función

⚙ Automatización

El script maestro `dulcenavidad.ps1` ejecuta el flujo de validación industrial en tres fases:

1. **Simetría binaria** — `bash scripts/comprobar-estructura` (main ↔ solutions).
2. **Modo reto** — `mvn clean test` (256 tests) con corte si falla.
3. **Modo soluciones** — `mvn -Psoluciones test`, con salida `BUILD SUCCESS`.

```
bash scripts/comprobar-estructura
mvn clean test
mvn -Psoluciones test
```